

First exercise in ML: building a classifier

Goal: Create a **classifier** using either a shallow Neural Network (such as MLP with a single hidden layer) or a Deep Network. The classifier can take as input some user generated data, e.g. a partition of the $[0,1] \times [0,1]$ square ($x_1 > x_2^2$).

Use Keras docs as needed <https://keras.io/api/>.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from keras.layers import Input, Dense # This is the bare minimum to build an MLP
from keras.models import Model
```

Generate some data.

```
In [2]: from math import *
# Fixing random state for reproducibility
np.random.seed(123)

# You can try changing this. Keep in mind that x1 and x2 are numpy arrays.
theFunction = lambda x1,x2 : np.sin(x1*20) > x2*x2 - x1

# Generating some random data
N=3000
x1 = 1.0 * np.random.rand(N)
x2 = 1.0 * np.random.rand(N)
y=theFunction(x1,x2)

# Creating a matrix with two columns one with data from x1, the other from x2
print("x1[:10] = ", x1[:10])
print("x2[:10] = ", x2[:10])
X=np.stack((x1,x2),axis=1)

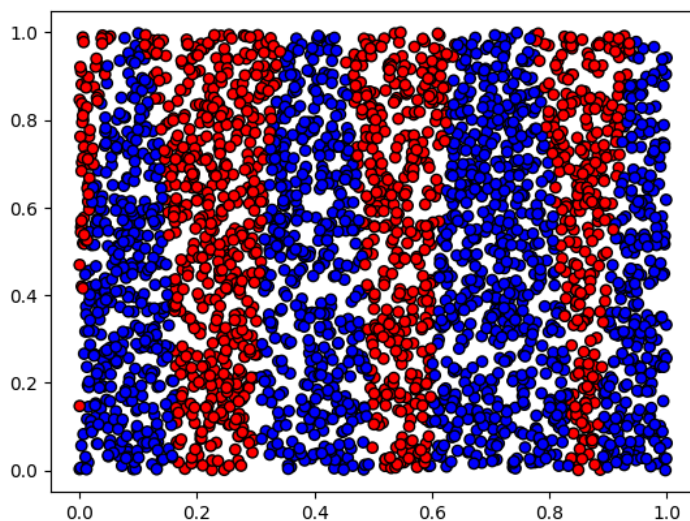
# Printing the first 10 entries of X and y
print("X[:10,:] = ", X[:10,:])
print("y[:20] = ", y[:20])
print(X.shape)

x1[:10] = [0.69646919 0.28613933 0.22685145 0.55131477 0.71946897 0.42310646
0.9807642 0.68482974 0.4809319 0.39211752]
x2[:10] = [0.96942611 0.72178369 0.18189166 0.88202837 0.47702151 0.84685675
0.83090753 0.01735057 0.23366189 0.91109161]
X[:10,:] = [[0.69646919 0.96942611]
[0.28613933 0.72178369]
[0.22685145 0.18189166]
[0.55131477 0.88202837]
[0.71946897 0.47702151]
[0.42310646 0.84685675]
[0.9807642 0.83090753]
[0.68482974 0.01735057]
[0.4809319 0.23366189]
[0.39211752 0.91109161]]
y[:20] = [ True False False False  True  True  True  True  True  True  True
 True  True  True  True False False False False]
(3000, 2)
```

Now let's look at the data we generated (visualizing it using a colormap). Does it look like you expect?

```
In [3]: cm_bright = ListedColormap(['#FF0000', '#0000FF'])
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cm_bright, edgecolors='k')
```

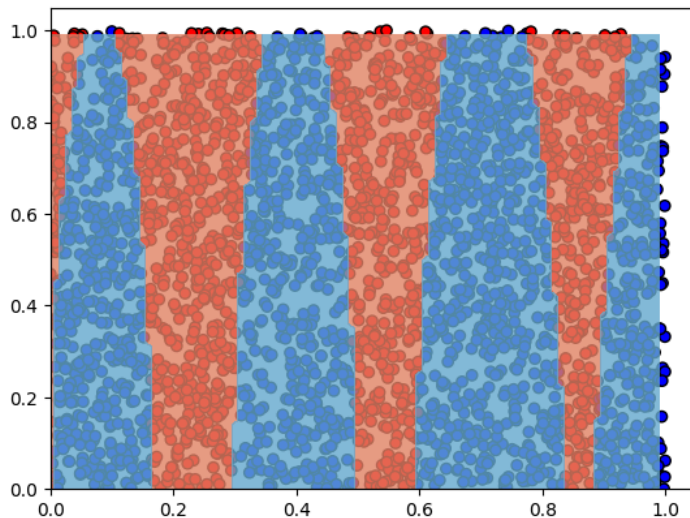
```
Out[3]: <matplotlib.collections.PathCollection at 0x797b46320b30>
```



Let's compare it with a contour map made using directly the generating function (remember, in real case world we will not know such function).

```
In [4]: xx1, xx2 = np.meshgrid(np.arange(0, 1, 0.01), np.arange(0, 1, 0.01))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cm_bright, edgecolors='k')
plt.contourf(xx1, xx2, (theFunction(xx1,xx2)).reshape(xx1.shape), cmap=plt.cm.RdBu, alpha=.8)
```

```
Out[4]: <matplotlib.contour.QuadContourSet at 0x797b4636a960>
```



This was just visualization. Now we want to see if a NN can distinguish "blue dots" from "red dots". Let's start with an MLP with 500 neurons.

Note on `Input(shape=(2,))` :

- **Focus on the single event:** The shape specifies the number of features per sample (in this case, 2), entirely ignoring the total dataset size (e.g. 3000).
- **The hidden batch dimension:** For computational efficiency on GPUs/CPU's, the network doesn't process data event-by-event, nor all at once. It uses "batches" (e.g. 32 samples at a time). Thus, the actual tensor shape during training will be for example `(32, 2)`.
- **Debugging tip:** If you ever get dimension mismatch errors mentioning an unexpected number like `32`, remember it's likely the batch size Keras is using under the hood.
- **Syntax details:** The trailing comma in `(2,)` is mandatory in Python to define a single-element tuple; otherwise, Python interprets it as a simple integer inside parentheses.

```
In [5]: # What is the shape that our input data has?
inputs=Input(shape=(2,))

# Creating a new dense layer with 500 nodes taking "inputs" as input. Why do we use the relu as the most appropriate activation function?
hidden=Dense(500, activation="relu")(inputs)

# Creating the output node. Why do we use the sigmoid as the most appropriate activation function?
outputs = Dense(1, activation='sigmoid')(hidden)
model = Model(inputs=inputs, outputs=outputs)
model.compile(loss='binary_crossentropy', optimizer='adam')

model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 2)	0
dense (Dense)	(None, 500)	1,500
dense_1 (Dense)	(None, 1)	501

Total params: 2,001 (7.82 KB)

Trainable params: 2,001 (7.82 KB)

Non-trainable params: 0 (0.00 B)

Using the `model.fit` function, divide the dataset in training dataset, validation and test dataset, then fit. `validation_split=0.5` divides the data in 50% training+validation and 50% test.

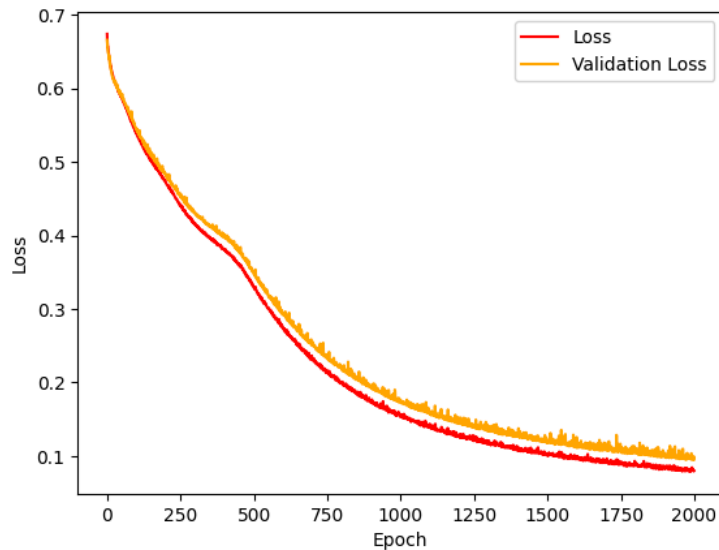
Also, you can set the `verbose` variable to 1 in order to have a deeper look at the training process. If you do so, a loading bar will appear, loading from 0 to 47 for each epoch of the processes (do it, this will all become clearer). That number, 47, is the number of training events per epoch, which is easily calculated. It is just the total number of training points (1500) divided by the batch size (32).

```
In [6]: # Using the Keras function fit to train the model
train_output=model.fit(X, y, validation_split=0.5, epochs=2000, verbose=0)
```

The `train_output` object contains a dictionary with all the important information about the training history: in the `train_output.history` dictionary you can find the training loss and validation loss per epoch. Let's try to plot them and compare them to evaluate the training.

```
In [15]: print(train_output.history.keys())
plt.plot(train_output.history['loss'], color="red", label="Loss")
plt.plot(train_output.history['val_loss'], color="orange", label="Validation Loss")
plt.legend()
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

```
dict_keys(['loss', 'val_loss'])
```

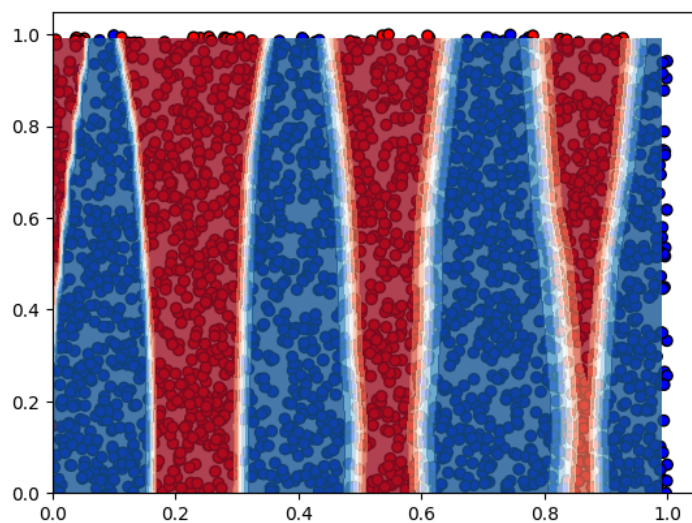


Now let's try to evaluate our model on the xx1,xx2 points we created earlier)

```
In [8]: testSet=np.stack((xx1.flatten(), xx2.flatten()), axis=-1)
pred=model.predict(testSet)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cm_bright, edgecolors='k')
plt.contourf(xx1, xx2, pred.reshape(xx1.shape), cmap=plt.cm.RdBu, alpha=.8)
```

```
313/313 ————— 1s 2ms/step
```

```
Out[8]: <matplotlib.contour.QuadContourSet at 0x797aeebb8950>
```



As you can see our NN works pretty well, even though the training took some time.

Now let's try to create a deep network instead with about the same number of parameters as the MLP above. Let's try with 4 hidden layers with 15 neurons each (so that every layer will add about 200 parameters).

```
In [12]: # Creating a DNN with 4 hidden Layers with 15 neurons each
inputs=Input(shape=(2,))

# Note: the hidden variable name is recycled, just for readability. You could have called each layer hidden1, hidden2 etc...
hidden = Dense(15, activation='relu')(inputs)
hidden = Dense(15, activation='relu')(hidden)
hidden = Dense(15, activation='relu')(hidden)
hidden = Dense(15, activation='relu')(hidden)
outputs = Dense(1, activation='sigmoid')(hidden)

deepmodel = Model(inputs=inputs, outputs=outputs)
deepmodel.compile(loss='binary_crossentropy', optimizer='adam')
deepmodel.summary()
```

```
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 2)	0
dense_2 (Dense)	(None, 15)	45
dense_3 (Dense)	(None, 15)	240
dense_4 (Dense)	(None, 15)	240
dense_5 (Dense)	(None, 15)	240
dense_6 (Dense)	(None, 1)	16

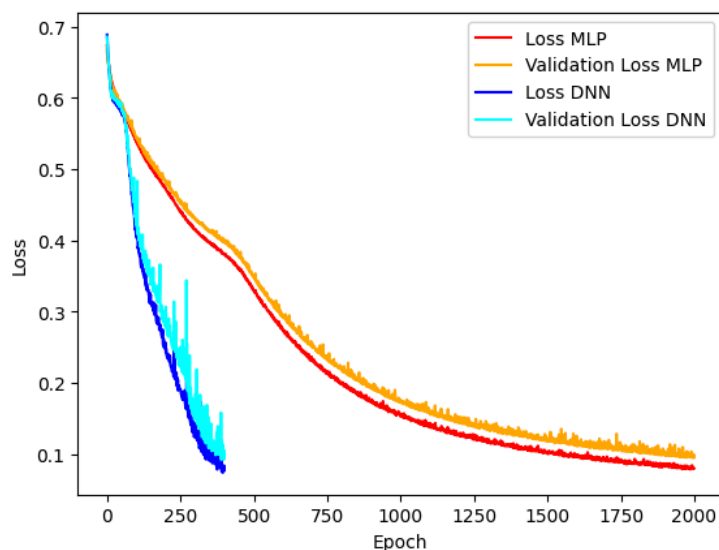
Total params: 781 (3.05 KB)

Trainable params: 781 (3.05 KB)

Non-trainable params: 0 (0.00 B)

```
In [16]: deep_train_output=deepmodel.fit(X, y, validation_split=0.5, epochs=400, verbose=0)
```

```
In [17]: plt.plot(train_output.history['loss'], color="red", label="Loss MLP")
plt.plot(train_output.history['val_loss'], color="orange", label="Validation Loss MLP")
plt.plot(deep_train_output.history["loss"], color="blue", label="Loss DNN")
plt.plot(deep_train_output.history["val_loss"], color="cyan", label="Validation Loss DNN")
plt.legend()
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```



As you can see from the graph, in about 400 epochs the DNN has surpassed the loss that the MLP had reached in 2000 epochs. Take a look below to see that the classification problem is resolved with good precision.

```
In [18]: testSet=np.stack((xx1.flatten(), xx2.flatten()), axis=-1)
pred=deepmodel.predict(testSet)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cm_bright, edgecolors='k')
plt.contourf(xx1, xx2, pred.reshape(xx1.shape), cmap=plt.cm.RdBu, alpha=.8)
```

313/313 ————— 1s 2ms/step

```
Out[18]: <matplotlib.contour.QuadContourSet at 0x797a8d97d430>
```

